Чтение и запись текстового файла

Цель работы

Научиться безопасно работать с текстовыми файлами в Python: создавать файл в собственной рабочей папке, записывать и дополнять текст, читать содержимое целиком и построчно, а также различать основные режимы открытия файла.

Среда выполнения: Python 3.x, Visual Studio Community. Все файлы создаются только в рабочей папке проекта.

1. Как программа обращается к текстовому файлу

Текстовый файл хранит последовательность символов: буквы, цифры, пробелы и переводы строк. Чтобы программа могла записать или прочитать данные, ей нужно знать **путь к файлу** и режим работы с ним.

В этой работе файл будет находиться в рабочей папке проекта. Это безопасно: программа не изменяет системные каталоги Windows и не требует запуска от имени администратора.

Сначала определим рабочую папку и будущий путь к файлу. Модуль pathlib позволяет записать путь понятно и без ручного ввода длинной строки.

Пример: определяем рабочую папку и путь к файлу

```
from pathlib import Path


work_folder = Path.cwd()
file_path = work_folder / "notes.txt"


print("Рабочая папка:", work_folder)
print("Будущий файл:", file_path)
```

Пример результата в консоли

```
Рабочая папка: C:\Users\Student\source\repos\TextFilesPractice
Будущий файл: C:\Users\Student\source\repos\TextFilesPractice\notes.txt
```

На разных компьютерах путь будет отличаться. Важно не запоминать путь из примера, а смотреть на тот путь, который вывела именно ваша программа.

ЗАДАНИЕ 1. Подготовка рабочей папки

Создайте в Visual Studio Community проект Python с именем TextFilesPractice. Вставьте приведённый выше пример и запустите его.

- Убедитесь, что программа выводит рабочую папку проекта.
- Найдите в выводе полный путь к будущему файлу notes.txt.
- Сам файл notes.txt пока создавать вручную не нужно — его создаст программа.

2. Открытие файла и режимы работы

Файл в Python открывается функцией open(). В скобках указывают путь, режим работы и кодировку текста. Для учебной работы используем UTF-8, чтобы русские буквы записывались и читались корректно.

Общая форма открытия текстового файла

```
with open(file_path, "w", encoding="utf-8") as file:
    # команды для работы с файлом находятся внутри блока with
    pass
```

Конструкция **with** открывает файл перед выполнением вложенных команд и автоматически закрывает его после выхода из блока. Это надёжнее, чем открывать файл и рассчитывать, что программа не забудет закрыть его вручную.

Режим	Назначение	Если файла нет	Что будет с содержимым
r	чтение	ошибка открытия	не изменяется
w	запись	файл создаётся	старое содержимое заменяется
a	дополнение	файл создаётся	новый текст добавляется в конец

Важно

Режим w нужно использовать осознанно: если файл уже существует, его прежнее содержимое будет удалено и заменено новым. Для добавления текста в конец существующего файла используется режим a.

3. Запись первой строки

Начнём с самого простого случая: программа создаёт файл notes.txt и записывает в него одну строку. Метод write() получает строку и помещает её в открытый файл.

Пример: создаём файл и записываем одну строку

```
from pathlib import Path

file_path = Path.cwd() / "notes.txt"

with open(file_path, "w", encoding="utf-8") as file:
    file.write("Первая запись в текстовом файле.\n")

print("Файл создан:", file_path)
```

Результат в файле notes.txt

```
Первая запись в текстовом файле.
```

После выхода из блока with файл закрывается автоматически. Символ \n в конце строки обозначает переход на новую строку. На экране он не отображается как два символа — в файле появляется настоящий перенос строки.

ЗАДАНИЕ 2. Первая собственная запись

Создайте файл first_record.txt и запишите в него одну строку: «Работа с текстовым файлом начата.»

- Используйте режим w и кодировку utf-8.
- После записи выведите в консоль полный путь к созданному файлу.
- Откройте файл обычным способом и убедитесь, что строка действительно сохранена.

4. Запись нескольких строк

Чтобы в файле было несколько строк, после каждой записи можно добавлять символ перехода на новую строку \n. Несколько вызовов write() последовательно записывают данные в один открытый файл.

Пример: записываем три строки

```
from pathlib import Path

file_path = Path.cwd() / "notes.txt"

with open(file_path, "w", encoding="utf-8") as file:
    file.write("Имя: Алексей\n")
    file.write("Группа: ИСИП\n")
    file.write("Язык: Python\n")

print("Три строки записаны.")
```

Содержимое notes.txt после запуска

Имя: Алексей
Группа: ИСИП
Язык: Python

Режим w снова создаёт содержимое файла заново. Поэтому после запуска этого примера прежняя строка «Первая запись в текстовом файле.» исчезнет и будет заменена тремя новыми строками.

ЗАДАНИЕ 3. Файл с несколькими строками

Создайте файл profile.txt и запишите в него три строки со своими данными.

- Первая строка — имя.
- Вторая строка — группа.
- Третья строка — название языка программирования Python.
- Каждая запись должна находиться на отдельной строке.

5. Чтение всего содержимого файла

Для чтения открываем существующий файл в режиме r. Метод read() считывает всё содержимое файла и возвращает его как одну строку Python. Эту строку можно сохранить в переменную, вывести или обработать.

Пример: читаем notes.txt целиком

```
from pathlib import Path

file_path = Path.cwd() / "notes.txt"

with open(file_path, "r", encoding="utf-8") as file:
    text = file.read()

print("Содержимое файла:")
print(text)
```

Результат в консоли

Содержимое файла:
Имя: Алексей
Группа: ИСИП
Язык: Python

После выполнения read() переменная text содержит весь текст файла. Команда print(text) выводит его с теми же переносами строк, которые были сохранены при записи.

ЗАДАНИЕ 4. Чтение собственного файла

Откройте созданный файл profile.txt в режиме r, прочитайте его методом read() и выведите содержимое в консоль.

- Сохраните прочитанный текст в переменную.
- После вывода текста дополнительно выведите количество символов с помощью len().
- Сравните вывод программы с реальным содержимым profile.txt.

6. Чтение файла построчно

Иногда не нужно загружать и выводить весь текст сразу. Открытый файл можно перебирать циклом for: на каждой итерации переменная line получает одну очередную строку.

Пример: читаем и нумеруем строки

```
from pathlib import Path


file_path = Path.cwd() / "notes.txt"
line_number = 1


with open(file_path, "r", encoding="utf-8") as file:
    for line in file:
        print(line_number, "-", line.strip())
        line_number += 1
```

Результат в консоли

```
1 - Имя: Алексей
2 - Группа: ИСИП
3 - Язык: Python
```

Метод strip() здесь убирает символ перехода на новую строку с конца прочитанной строки. Благодаря этому print() не создаёт лишние пустые строки между записями.

ЗАДАНИЕ 5. Построчный просмотр

Прочитайте profile.txt построчно и выведите каждую строку с её номером.

- Начните нумерацию с 1.
- После цикла выведите сообщение с количеством прочитанных строк.
- Проверьте, совпадает ли это число с количеством строк в самом файле.

7. Добавление текста без удаления старых данных

Если нужно сохранить существующие данные и добавить новую запись в конец файла, используется режим a. В этом режиме Python не очищает файл перед записью.

Пример: добавляем новую строку в конец файла

```
from pathlib import Path

file_path = Path.cwd() / "notes.txt"

with open(file_path, "a", encoding="utf-8") as file:
    file.write("Статус: работа выполнена\n")

with open(file_path, "r", encoding="utf-8") as file:
    text = file.read()

print(text)
```

Содержимое после дополнения

```
Имя: Алексей
Группа: ИСИП
Язык: Python
Статус: работа выполнена
```

Обратите внимание: вторая часть программы снова открывает тот же файл, но уже в режиме r. Поэтому мы сразу проверяем результат — сначала дополняем файл, затем читаем его и убеждаемся, что прежние строки сохранились.

ЗАДАНИЕ 6. Добавление пользовательской строки

Дополните profile.txt новой строкой, введённой с клавиатуры.

- Получите текст командой input().
- Откройте profile.txt в режиме a и добавьте введённый текст в конец файла.
- Не забудьте добавить \n, чтобы следующая возможная запись начиналась с новой строки.
- После дополнения откройте файл в режиме r и выведите итоговое содержимое.

8. Что произойдёт при неправильном выборе режима

Главная практическая разница между w и a проявляется не в синтаксисе, а в результате. Оба режима позволяют записывать текст, но по-разному обращаются с тем, что уже находится в файле.

Запомните

w — начать содержимое заново. a — сохранить старые данные и дописать новые. r — только прочитать существующий файл. Перед запуском программы всегда проверяйте, какой режим указан в open().

Мини-эксперимент: сравнение w и a

```
from pathlib import Path

file_path = Path.cwd() / "mode_test.txt"

with open(file_path, "w", encoding="utf-8") as file:
    file.write("Строка 1\n")

with open(file_path, "a", encoding="utf-8") as file:
    file.write("Строка 2\n")

with open(file_path, "r", encoding="utf-8") as file:
    print(file.read())
```

Результат

```
Строка 1
Строка 2
```

Если во втором блоке заменить a на w, после запуска в файле останется только «Строка 2». Это наглядно показывает, почему режим записи нужно выбирать до открытия файла.

9. Итоговый пример: небольшой текстовый журнал

Теперь объединим основные действия в одной короткой программе. Она создаёт журнал, записывает начальную строку, получает новую заметку от пользователя, добавляет её и затем показывает весь файл.

Полный пример программы

```
from pathlib import Path

file_path = Path.cwd() / "journal.txt"

with open(file_path, "w", encoding="utf-8") as file:
    file.write("Учебный журнал\n")

note = input("Введите заметку: ")

with open(file_path, "a", encoding="utf-8") as file:
    file.write(note + "\n")

with open(file_path, "r", encoding="utf-8") as file:
    text = file.read()

print("\nСодержимое журнала:")
print(text)
```

Пример запуска

```
Введите заметку: Файл успешно создан и дополнен.


Содержимое журнала:
Учебный журнал
Файл успешно создан и дополнен.
```

Программа использует три отдельных открытия одного файла. Это сделано специально: каждый блок имеет одну понятную задачу — создать начальное содержимое, дополнить его и прочитать результат.

ЗАДАНИЕ 7. Итоговая самостоятельная работа

Создайте программу «Список событий» для файла events.txt. Используйте только действия, разобранные выше.

- В режиме w запишите заголовок «События программы» и две исходные строки.
- В режиме r прочитайте файл и покажите первоначальное содержимое.
- Запросите у пользователя ещё одно событие через input().
- В режиме a добавьте введённое событие в конец файла.
- Снова откройте файл в режиме r и выведите итоговое содержимое.
- Построчно посчитайте и выведите количество строк в итоговом файле.

10. Если результат отличается от ожидаемого

Если файл не появился

Посмотрите путь, который формирует Path.cwd(). Файл создаётся именно в этой рабочей папке. Также убедитесь, что блок with действительно был выполнен и программа завершилась без синтаксической ошибки.

Если старый текст исчез

Проверьте режим открытия. Повторное открытие существующего файла с "w" очищает прежнее содержимое. Для добавления новой записи без удаления старой используйте "a".

Если при чтении сообщается, что файл не найден

Сначала убедитесь, что файл уже был создан программой и что при записи и чтении используется одно и то же имя. В этой работе отдельная обработка исключений не требуется.

11. Что должно быть в отчёте

- тема и цель работы;
- исходный код итоговой программы «Список событий»;
- скриншот консоли, на котором видно первоначальное и итоговое содержимое файла;
- скриншот или показ файла events.txt с итоговыми строками;
- ответы на вопросы для проверки понимания;
- краткий вывод: чем отличаются режимы r, w и a и какой результат был получен.

12. Вопросы для проверки понимания

1. Для чего функции open() передаётся режим работы с файлом?
2. Что произойдёт с существующим содержимым файла при открытии в режиме w?
3. Чем режим a отличается от режима w?
4. Для чего в примерах используется encoding="utf-8"?
5. Что делает конструкция with при работе с файлом?
6. Чем чтение методом read() отличается от перебора файла циклом for?
7. Зачем при записи нескольких строк используется \n?
8. Почему файл в этой практической создаётся только в рабочей папке проекта?

13. Результат выполнения

- созданы и проверены текстовые файлы в рабочей папке проекта;
- выполнена запись одной и нескольких строк;
- файл прочитан целиком и построчно;
- выполнено дополнение существующего файла без удаления прежнего содержимого;
- создана итоговая программа «Список событий», использующая режимы w, r и a;
- в отчёте зафиксированы код, результаты запуска, ответы и краткий вывод.

Критерий готовности

Работа завершена, если программа запускается без синтаксических ошибок, создаёт events.txt в рабочей папке, сохраняет исходные строки, добавляет новую строку без удаления старых данных, читает итоговое содержимое и выводит количество строк, а в отчёте присутствуют требуемые подтверждения.